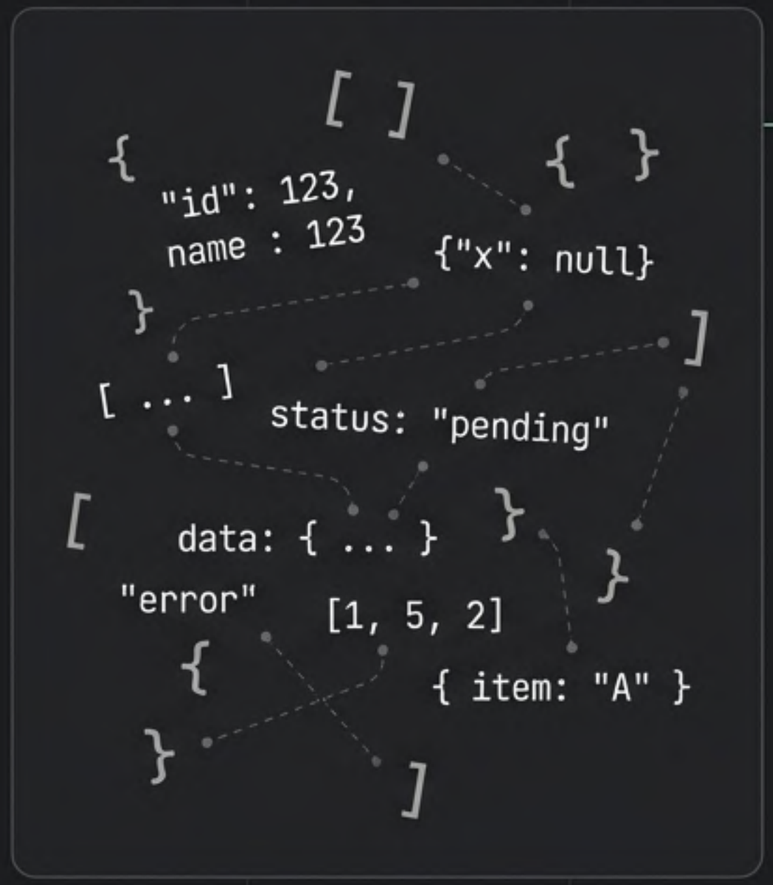
Maestría en Manipulación de Datos con JavaScript

De mutaciones básicas a pipelines funcionales.




```
// Datos crudos (mutables, sin procesar)
```

```
processData = (raw) =>
  raw.filter(...).map(...);
```

```
[
  {
    "id": 123,
    "name": "Item A",
    "status": "processed",
    "value": 10.5
  },
  {
    "id": 124,
    "name": "Item B",
    "status": "processed",
    "value": 20.0
  },
  // ... más elementos limpios
]
```

```
// Salida inmutable y estructurada
```

NotebookLM

El Fundamento: Funciones Flecha

Antes: Función Tradicional

```
let sum = function(a, b) {
  return a + b;
};
```

Después: Función Flecha

```
let sum = (a, b) => a + b;
```

Sintaxis compacta

Retorno implícito

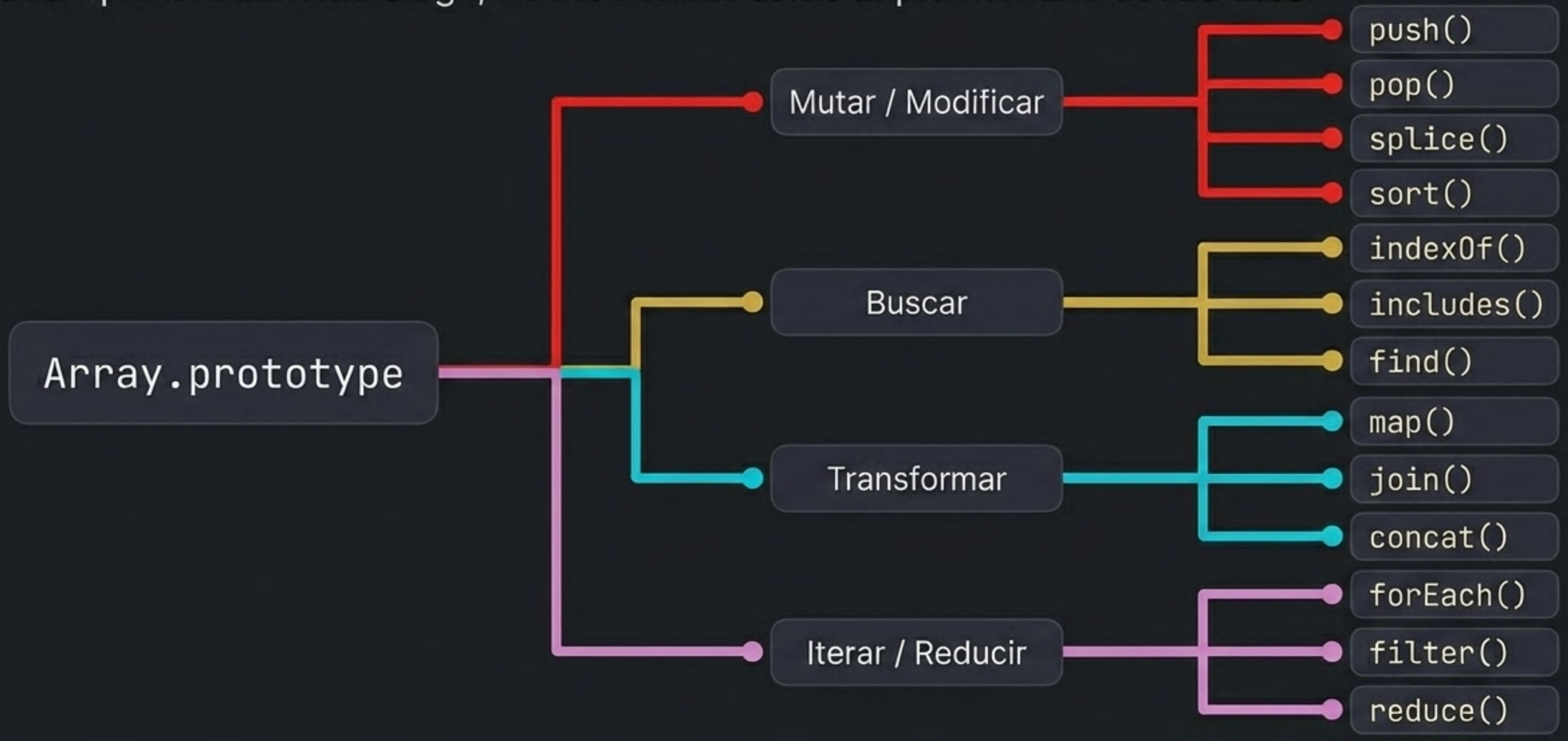
```
n => n * 2
```

Sin paréntesis para un solo argumento

NotebookLM

El Espectro de los Arreglos

Saber qué herramienta elegir, no memorizar cómo implementarla desde cero.

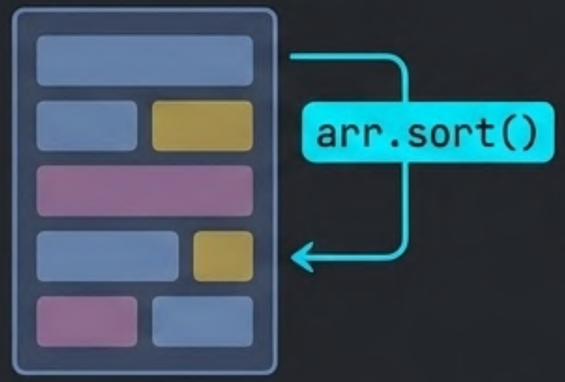


NotebookLM

El Peligro Oculto: Mutación vs. Copia

Métodos Destructivos

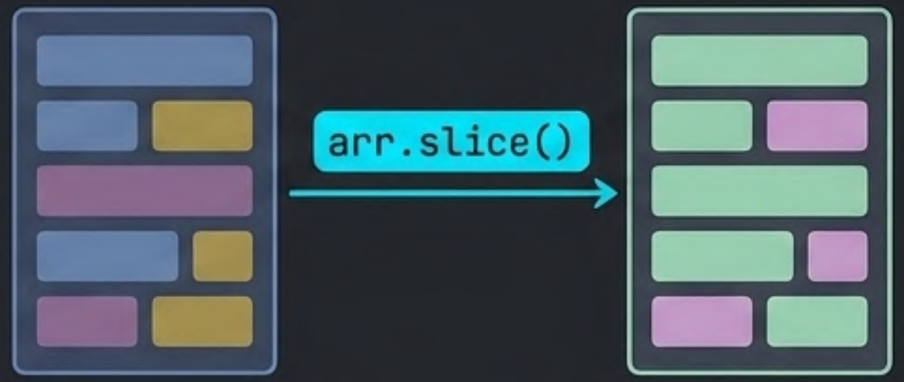
- push
- splice
- sort
- reverse



Modifica la fuente original

Métodos Inmutables

- concat
- slice
- map
- filter



Retorna una nueva matriz pura

NotebookLM

La Navaja Suiza vs. El Escáner: splice() y slice()

```
splice(start, deleteCount, items)
```

MUTA el arreglo original.

```
slice(start, end)
```

RETORNA una copia superficial (*shallow copy*).

NotebookLM

Búsqueda Eficiente: Primitivos vs. Objetos

Datos Primitivos

```
let arr = [1, 0, NaN];

arr.indexOf(NaN)
  ├──> -1 (Error de diseño antiguo)
  │
arr.includes(NaN)
  └──> true (Manejo correcto)
```

Arreglo de Objetos

```
users.find(user => user.id === 1)
```

Pasamos una función evaluadora que retorna el primer objeto donde la condición sea true.

No podemos usar includes para objetos debido a las referencias de memoria.

NotebookLM

El Salto Conceptual: Funciones de Orden Superior

Abstraemos la acción, no solo el valor.

```
for (let i = 0; i <
arr.length; i++) {

  // lógica manual
}
```

Orden de acción

```
arr.forEach(acción)
```

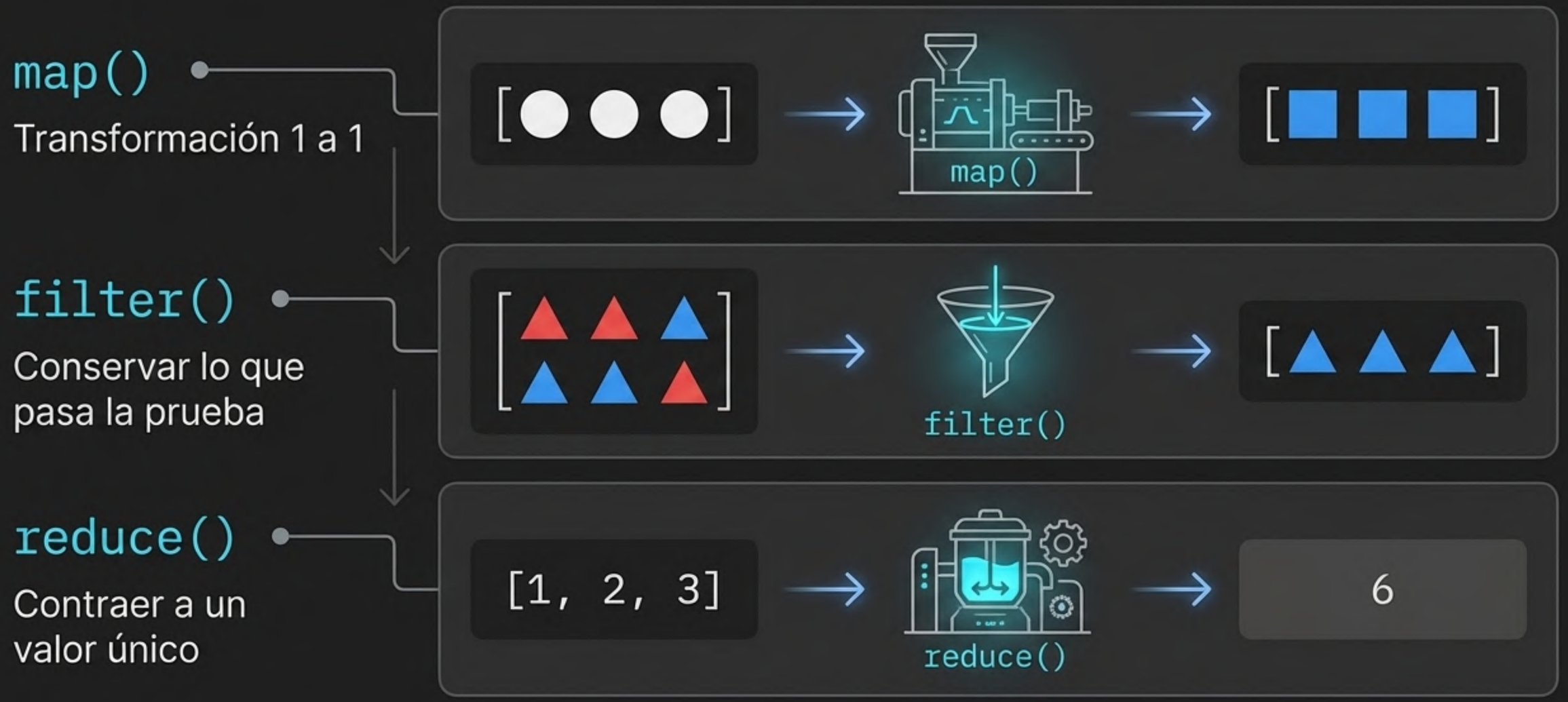
Iteración Manual
(Visual Noise)

Función de Orden Superior
(Abstracción Limpia)

La máquina maneja la iteración.
Tú solo pasas la función ('acción') que define qué hacer.

NotebookLM

La Trinidad de la Transformación de Datos

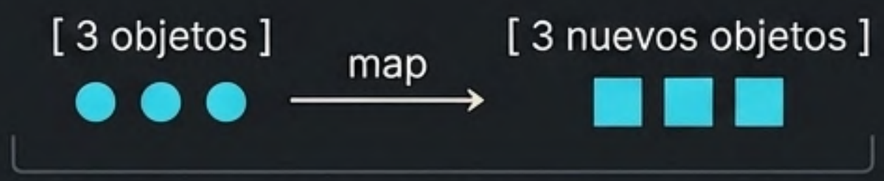


NotebookLM

map(): Mapeo Estructural

```
let usersMapped = users.map(user => ({
  fullName: `${user.name} ${user.surname}`,
  id: user.id
}));
```

⚠️ Cuidado con la sintaxis: JavaScript confunde las llaves {} del objeto literal con el inicio del cuerpo de la función flecha. Debes envolver el objeto en paréntesis ().

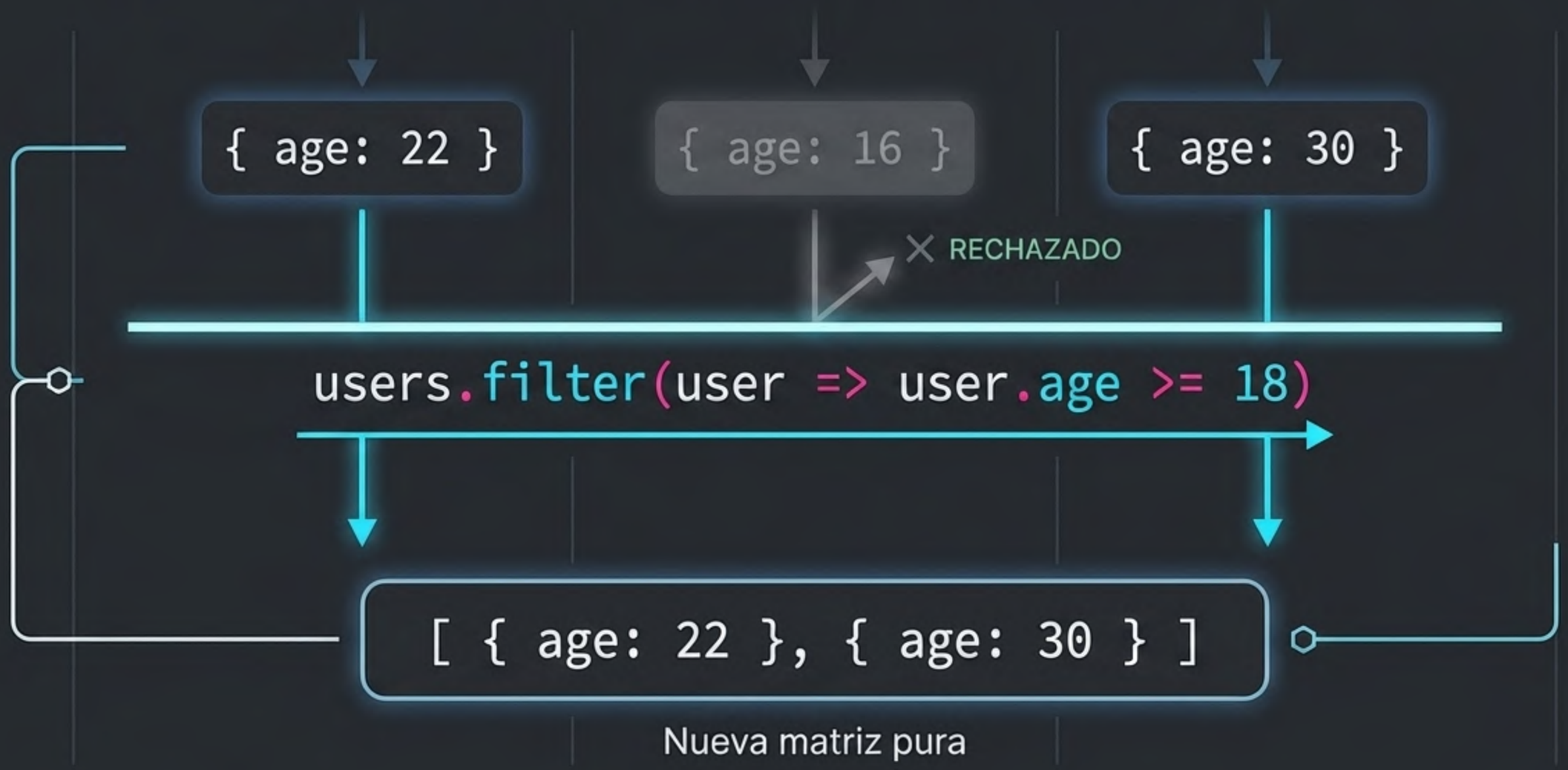


La longitud de salida siempre es igual a la entrada.

NotebookLM

filter(): La Barrera Condicional

Crea una matriz pura con los elementos sobrevivientes.



NotebookLM

reduce(): Anatomía del Acumulador

```
let sum = arr.reduce((acc, curr) => acc + curr, 0);
```

Valor Inicial

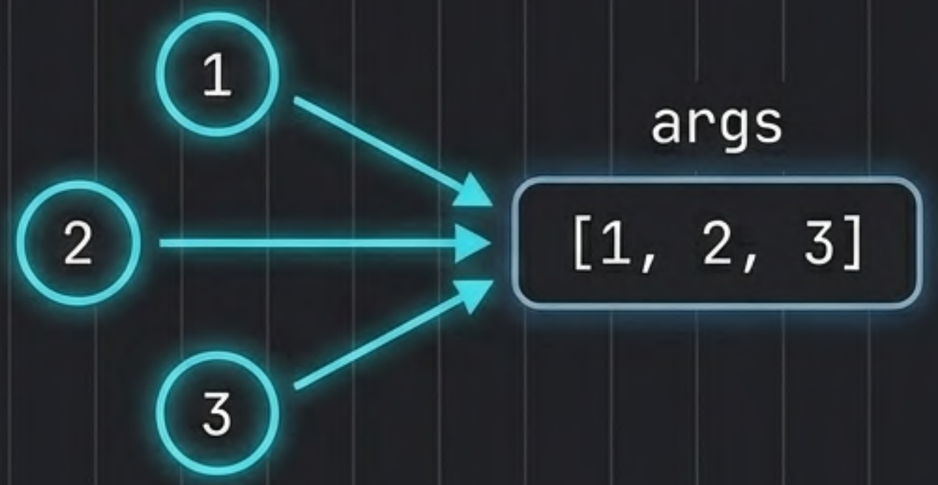
Iteración	acc (Acumulador)	curr (Actual)	Retorno
Llamada 1	0	1	1
Llamada 2	1	2	3
Llamada 3	3	3	6

NotebookLM

La Dualidad de los 3 Puntos: Rest vs. Spread

Rest Parameter

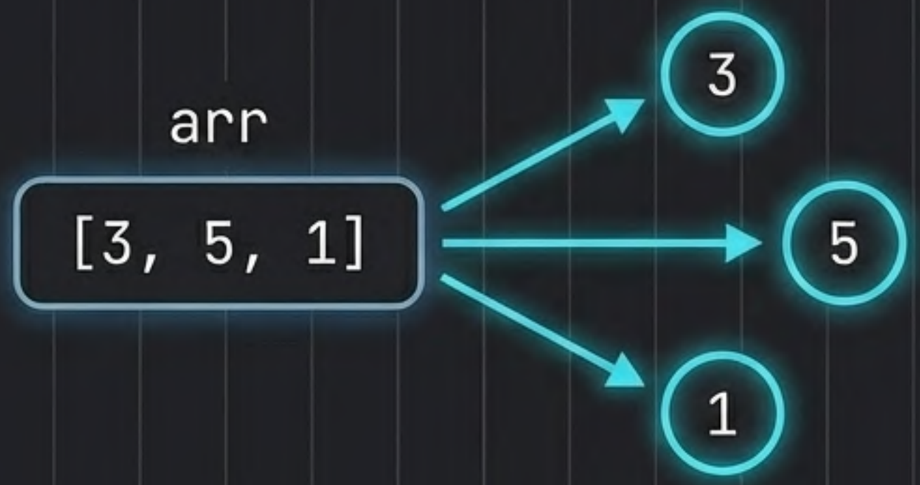
```
function sumAll(...args) {
}
```



Recolecta los parámetros restantes en un arreglo. Siempre va al final de la firma.

Spread Syntax

```
Math.max(...arr)
```



Expande un iterable (como un arreglo o string) en una lista de argumentos.

NotebookLM

Magia Diaria con Spread Syntax

Copia Superficial (Shallow Copy)

```
let arrCopy = [...arr];
```

Rápido, limpio, reemplaza a Object.assign().

Fusión de Arreglos

```
let merged = [0,
  ...arr1, 2, ...arr2];
```

Combina iterables e inyecta valores fijos en cualquier posición.

Conversión de Strings

```
let chars = [..."Hello"];
// ['H','e','l','l','o']
```

Resultado del iterador

Internamente usa iteradores, separando caracteres fácilmente.

NotebookLM

Síntesis: El Pipeline Funcional

Código declarativo: Decimos qué queremos, no cómo hacerlo paso a paso.

```
let averageActiveAge = users                                  [Objetos]
  .filter(u => u.isActive)      // 1. Embudo: Retiene vivos   [Objetos Activos]
  .map(u => u.age)              // 2. Molde: Extrae edades    [Números]
  .reduce((a, b) => a + b, 0)   // 3. Acumulador: Suma total  [Número Final]
  / users.length;
```

NotebookLM

Cheat Sheet Definitiva de Arreglos

Mantén los datos inmutables y deja que las funciones cuenten la historia.

¿Añadir o Quitar?

push / pop (final)

unshift / shift (inicio)

splice (cualquier punto)

¿Buscar?

indexOf / includes (primitivos)

find / findIndex (lógica/objetos)

¿Transformar?

map (modifica todos)

filter (retiene algunos)

sort (reordena in-place)

¿Reducir o Unir?

reduce (un solo valor)

[...a, ...b] o concat (fusión)

NotebookLM